

Mobile App Developer Assignment

Objective

Build a demo Android application that fetches, renders, and tracks **display banner ads** using the Osmos Ads system.

The goal is to evaluate your ability to:

- Work with API-driven ad responses
 - Render ads dynamically
 - Track impressions & clicks correctly
 - Build production-ready mobile components
-

Reference Documentation

Use the official docs:

- SDK Initialization
👉 <https://dev-hub.osmos.ai/docs/initialize-the-android-sdk-1>
 - Ad Fetching (Display Ads via AU)
👉 <https://dev-hub.osmos.ai/docs/ad-fetching-android-1>
 - Event Tracking
 - 👉 Impression: <https://dev-hub.osmos.ai/docs/register-events-android#ad-impression-event>
 - 👉 Click: <https://dev-hub.osmos.ai/docs/register-events-android#ad-click-event>
-

Problem Statement

Create a demo mobile app that:

- Fetches **display ads using Ad Unit (AU)**
 - Renders **banner image ads manually**
 - Fires **impression and click tracking events**
 - Handles failures and edge cases gracefully
-

Core Requirements

1. SDK Integration

- Integrate Osmos Android SDK
- Initialize SDK using:

```
clientId = 10088010
productAdsHost = "demo.o-s.io"
displayAdsHost = "demo-ba.o-s.io"
```

- Keep SDK logic modular
-

2. Fetch Display Ads (AU-based)

Use the following request:

```
"cliUbid"= "Any"
"pageType"= "demo_page"
"adUnit"= "banner_ads"
```

- Fetch ads from SDK / API
- Parse response and extract:

```
ads.banner_ads[0]
```

Key fields:

- `elements.value` → Image URL
 - `elements.destination_url` → Landing URL
 - `impression_tracking_url`
 - `click_tracking_url`
-

3. Render Banner Ad

- Display image using `ImageView`
- Maintain aspect ratio using:

- width
 - height
 - Provide UI:
 - Button → “Load Ad”
 - Container → Banner render area
-

4. Impression Tracking (Important)

Fire impression when:

👉 **At least 50% of the ad is visible on screen**

Requirements:

- Detect visibility
- Trigger only once per ad

Use:

- SDK event method (as per docs)
-

5. Click Handling

On ad click:

1. Open:

```
elements.destination_url
```

2. Fire click event:
 - SDK click event API
-

6. Event Logging

Track and log:

- Ad Loaded
- Ad Failed
- Impression Fired

- Click Fired

Use:

- Logcat
 - OR simple UI indicators
-

7. Error Handling & Resilience

Handle:

- SDK initialization failure
- No ads in response
- Network errors
- Invalid/missing fields

Ensure:

- No crashes
 - Show fallback UI:
👉 “Ad not available”
-

8. UX & Optimization

- Show loading state while fetching ad
 - Prevent duplicate requests
 - Handle:
 - Activity lifecycle
 - Screen rotation
-

Required Enhancements

- Add retry mechanism for ad fetching
 - Implement reusable visibility tracking helper (50% logic)
 - Add simple analytics/logging layer
-

Submission Guidelines

1. GitHub Repository

- Working Android project
 - Clear folder structure
-

2. README.md (Important)

Include:

- Setup steps
 - How ad fetching works
 - How impression logic is implemented (50% visibility)
 - How click tracking is handled
 - Architecture explanation
 - Assumptions made
 - Challenges faced
 - How to run the demo
-

3. Demo (Optional but Recommended)

Short recording showing:

- Ad loading
- Banner rendering
- Impression firing
- Click handling
- Error scenarios